GPT: the Decoder-Only Transformer

Contents

1. Overview . . . 2
2. Prerequisites . . . 3
3. From the transformer decoder to GPT . . . 3
4. Tokens . . . 5
5. From tokens to vectors: token and position embeddings . . . 6
6. Inside a GPT-2 block . . . 7
    - 6.1 Prenorm: LayerNorm at the input . . . 7
    - 6.2 The residual stream grows, and the final LayerNorm brings it back . . . 8
    - 6.3 Masked self-attention and the MLP . . . 9
    - 6.4 GELU . . . 10
7. A guess at every position . . . 11
    - 7.1 The tied unembedding . . . 11
    - 7.2 Every position predicts . . . 12
    - 7.3 Training: all positions at once . . . 13
    - 7.4 Generation: predict, pick, append, repeat . . . 14
8. Context size . . . 14
9. Counting the parameters . . . 15
    - 9.1 GPT-2 small, matrix by matrix . . . 16
    - 9.2 GPT-3 . . . 17
10. Summary . . . 19
11. Sources . . . 19
12. Key terms . . . 20

Where this fits: Pipeline map step 8 (Model). See the Course map.

0 Foundations	1 Frame	2 Get data	3 Understand	4 Clean	5 Features	6 Reduce
7 Split	**8 Model**	9 Evaluate	10 Tune	11 Deploy	12 Test	13 Monitor

- **Builds on:** GELU activation; Transformer; Masked self-attention; Transformer decoder.
- **Leads to:** Unembedding, logits, temperature and sampling; MLP blocks as fact storage.

1. Overview

Key point: GPT (G-855) is a transformer with the encoder removed: a stack of identical blocks that reads a text and, in one pass, guesses the next **token** (G-1981) at **every** position.

Each block has two parts: **masked self-attention** (G-1172), then an MLP. Each part adds its result to the token's vector. Training improves all the guesses at once. Generation keeps only the last guess, appends it, and runs the pass again. We load the released GPT-2 small (124,439,808 parameters) and run it in plain NumPy to check every claim.

The decoder of the translation transformer (from a black box to the decoder block) has three sub-layers: masked self-attention, cross-attention to the encoder, and a feed-forward network. GPT keeps the decoder and drops the encoder. With no encoder there is nothing to cross-attend to, so the cross-attention goes too. Radford et al. (2018, §4.1) describe their model, a **decoder-only transformer** (G-565), as a "12-layer decoder-only transformer with masked self-attention heads".

A **token** is a piece of text, a word or part of a word (section 4). The **MLP** (multi-layer perceptron) is the feed-forward network of the transformer: a small two-layer network applied to each token's vector on its own.

Figure 1 follows one real pass of GPT-2 small on "Steve Jobs was the founder of". Watch three things:

1. Each token takes its row of the embedding matrix (drawn as an upright bar).
2. In attention, arrows only point **right**: a token reads only the tokens before it.
3. At the top, a guess appears above **every** token, not just the last one. The last guess, " Apple", is the one used for generation.

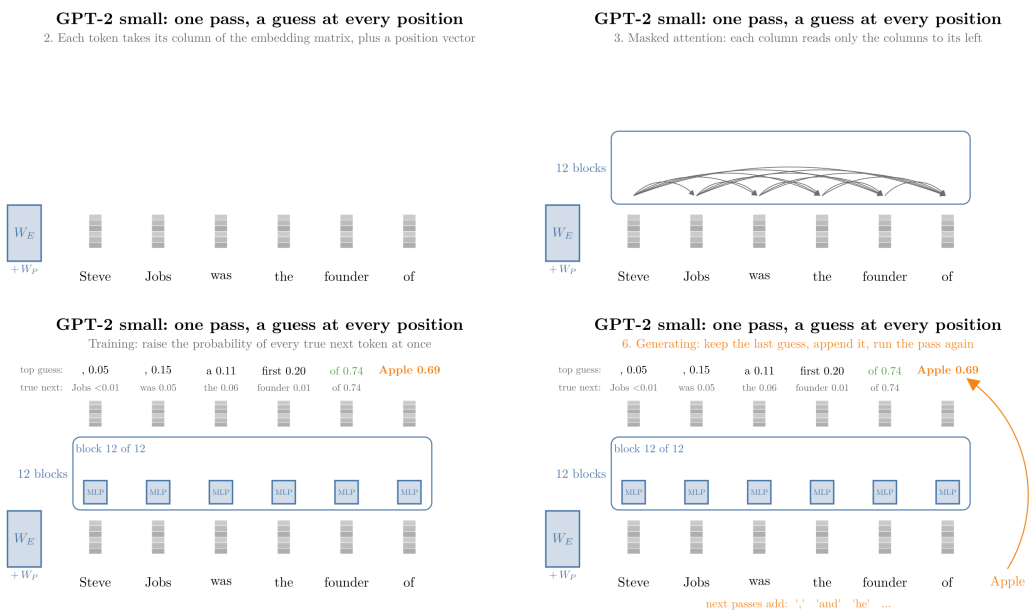


Figure 1: One pass of GPT-2 small (real weights). Blue: learned weights. Grey: the data flowing through. Top row: the model's top guess for the next token at each position, with its probability. Second row: the true next token and the probability the model gave it (green = guessed right)

The order of this Note follows the data through the model: tokens (section 4), vectors (section 5), the blocks (section 6), the guesses at every position (section 7), how much text fits (section 8), and finally where the parameters sit (section 9).

2. Prerequisites

- Transformer decoder: the decoder block, the output layer (linear and softmax), training on every position in one pass.
- **Masked self-attention** (G-1172): the causal mask; why one pass equals many steps.
- Transformer inference: generation one **token** (G-1981) at a time.
- Transformer encoder: residual connections and the feed-forward network.
- **Layer normalisation** (G-1054): LayerNorm, and the prenorm and postnorm placements.
- **Positional encoding** (G-1528): why the model needs positions.
- **Multi-head attention** (G-1268): heads, W_Q, W_K, W_V, W_O, and the $4d^2 + 4d$ count.
- History of LLMs: where **GPT** (G-855), GPT-2 and GPT-3 come from.

3. From the transformer decoder to GPT

> **Key point:** GPT keeps the decoder's **masked self-attention** (G-1172) and feed-forward network and removes the cross-attention, because there is no encoder to attend to. Each GPT block has two sub-layers instead of three.

The translation transformer needs two inputs: the source sentence (read by the encoder) and the target written so far (read by the decoder). A language model has only one input: the text so far. It predicts the next **token** (G-1981) of that same text. So one stack is enough. Radford et al. (2018, §3.1) train it to maximise the likelihood of each token given the k tokens before it:

$$L_1(U) = \sum_i \log P(u_i \mid u_{i-k}, \ldots, u_{i-1})$$

Here u_i is the i-th token of the text, k is how many earlier tokens the model sees, the bar $\mid$ reads "given", and $\sum_i$ adds over every position. For one position, say the model gives the token "sat" the probability 0.25 after seeing "the cat":

$$\log P(\text{sat} \mid \text{the cat}) = \log 0.25 = -1.39$$

Training raises this sum, so the probabilities of the true next tokens rise.

LayerNorm (how layer normalisation works) rescales one token's vector to mean 0 and standard deviation 1. **Postnorm** means it comes after each addition; **prenorm** means it comes at the input of each sub-layer.

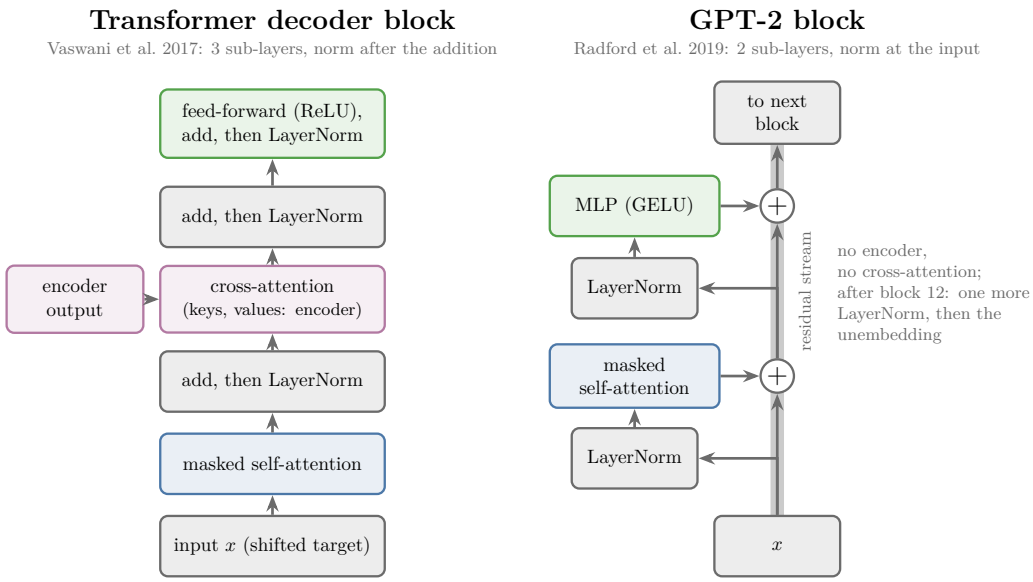


Figure 2: Left: the decoder block of the translation transformer (3 sub-layers, LayerNorm after each addition). Right: a GPT-2 block (2 sub-layers, LayerNorm at the input of each, the residual stream passes straight through)

	Transformer decoder (Vaswani et al. 2017)	GPT-2 block (Radford et al. 2019)
Sub-layers	masked self-attention, cross-attention, feed-forward	masked self-attention, MLP (feed-forward)
Encoder	yes	none
Where LayerNorm sits	after each addition (postnorm)	at the input of each sub-layer (prenorm), plus one after the last block
Activation in the feed-forward network	ReLU	GELU
Positions	fixed sine–cosine vectors	a learned table
Output layer	tied to the embeddings (Vaswani et al. 2017, §3.4)	the token embedding matrix itself (tied)

Figure 2 shows the two blocks side by side. The rest of this Note goes through the right-hand column, row by row, on the real GPT-2 small. The two names "MLP" and "feed-forward network" mean the same block; GPT-2's weights call it `mlp`.

4. Tokens

> **Key point:** GPT-2 reads text as **tokens** (G-1981), pieces of words from a fixed list of 50,257. Common words are one token; rare words are split into pieces.

A tokenizer cuts the text into pieces from a fixed vocabulary. GPT-2's vocabulary has 50,257 tokens (Radford et al. 2019, §2.3). It is built with **byte pair encoding** (G-335), which starts from single bytes and merges the most frequent pairs (Radford et al. 2019, §2.2). The Notebook runs GPT-2's own tokenizer:

Text	Tokens
Steve Jobs was the founder of	`Steve`, `Jobs`, `was`, `the`, `founder`, `of` (6 tokens)
Unbelievably, tokenization splits rare words.	`Un`, `bel`, `iev`, `ably`, `,`, `token`, `ization`, `splits`, `rare`, `words`, `.` (11 tokens)

The space before a word belongs to the token: " Jobs" with a space and "Jobs" without one are different tokens (Figure 3). In the other figures we draw the word without its space.

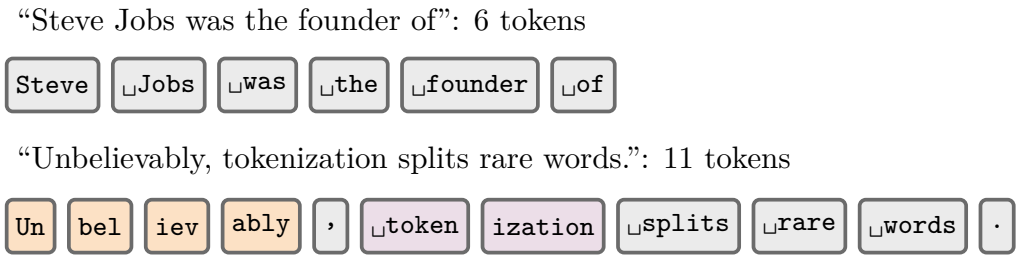


Figure 3: GPT-2's tokenizer on the two example texts. Each box is one token; the open-box sign marks the space that belongs to it. "Unbelievably" becomes 4 tokens and "tokenization" 2

5. From tokens to vectors: token and position embeddings

> **Key point:** Each **token** (G-1981)'s vector is its row of the **token embedding matrix** (G-1980) W_E (50,257 × 768) plus the row of the **position embedding matrix** (G-1526) W_P (1,024 × 768) for its place in the text. Both matrices are learned.

The first step is a lookup, as in turning words into numbers. Token number 19206, "Steve", takes row 19206 of W_E: 768 numbers. Then the vector for position 0 is added from W_P. For the token t_i at position i:

$$x_i = W_E[t_i] + W_P[i]$$

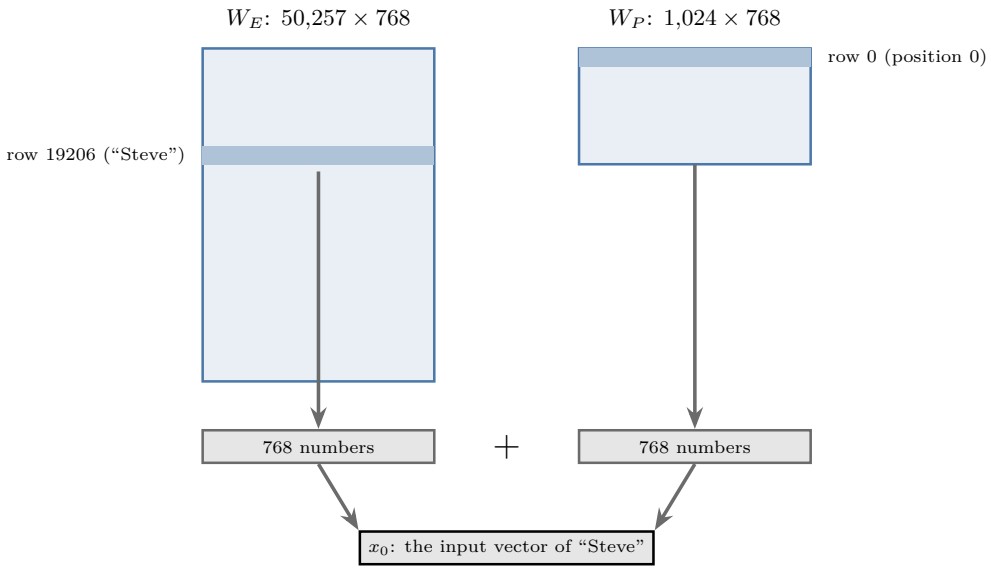


Figure 4: The first input vector of the pass: row 19206 of W_E (the token “Steve”) plus row 0 of W_P (position 0). Blue: learned weights; grey: data

Figure 4 shows the lookup and the addition for the first token.

GPT (G-855) uses **learned** positions: “We used learned position embeddings instead of the sinusoidal version proposed in the original work” (Radford et al. 2018, §4.1). W_P is just another weight matrix, trained with the rest. The swap costs little in quality: the original transformer paper had found that learned and sine–cosine positions give nearly identical results (Vaswani et al. 2017, Table 3, row E). Positions are needed at all because attention by itself ignores word order (why the transformer needs positions).

6. Inside a GPT-2 block

> **Key point:** A block reads the **token** (G-1981) vector through a LayerNorm, computes a change, and **adds** the change back, once for attention and once for the MLP. The vector that flows from block to block, never replaced, only added to, is called the **residual stream** (G-1683).

6.1 Prenorm: LayerNorm at the input

> **Key point:** GPT-2 moved each LayerNorm from after the addition to the input of the sub-layer, and added one more LayerNorm after the last block.

The paper states the change in one sentence: “Layer normalization (Ba et al., 2016) was moved to the input of each sub-block, similar to a pre-activation residual network (He et al., 2016) and an additional layer normalization was added after the final self-attention block” (Radford et al. 2019, §2.3). The released weights match: every block has `ln_1` (before

attention) and `ln_2` (before the MLP), and there is one `ln_f` after block 12. The placement is called **prenorm** (where LayerNorm sits).

Why move it? Xiong et al. (2020, abstract) showed that with the LayerNorm after each addition, the gradients near the output are large at the start of training, so the original transformer needed a slow learning-rate warm-up. With the LayerNorm at the input of each sub-layer, the gradients are well-behaved from the start, and the model trains without a warm-up and with less tuning.

1. **In words:** normalise a copy of the vector, let the sub-layer compute a change from the copy, and add the change to the original. The arrow $\leftarrow$ in the formula means "replace x by".
2. **Formula:** for each block, first attention, then the MLP:

$$x \leftarrow x + \mathrm{Attn}\Big(\mathrm{LN}_1(x)\Big)$$

$$x \leftarrow x + \mathrm{MLP}\Big(\mathrm{LN}_2(x)\Big)$$

   After block 12, the final LayerNorm:

$$h = \mathrm{LN}_{\mathrm{f}}(x)$$

3. **Example:** the Notebook normalises the stream vector of " of" before block 6 and gets mean 0.000 and standard deviation 1.000, whatever the stream's own size.

6.2 The residual stream grows, and the final LayerNorm brings it back

> **Key point:** In GPT-2 small the stream vector of " of" grows from length 4.9 to 400 over the 12 blocks, because each block adds and nothing rescales. The final LayerNorm brings it back to a fixed scale; without it, the prediction is lost.

In the prenorm formula only the copies are normalised; the stream x itself is never rescaled. Each block adds two vectors to it, so nothing stops its length from growing. Figure 5 measures this for the last **token** (G-1981), " of". The **length** of a vector is the length of its arrow: the square root of the sum of its squared entries. For the vector (3, 4):

$$\sqrt{3^2 + 4^2} = \sqrt{25} = 5$$

The stream vector has 768 entries, but its length is computed the same way.

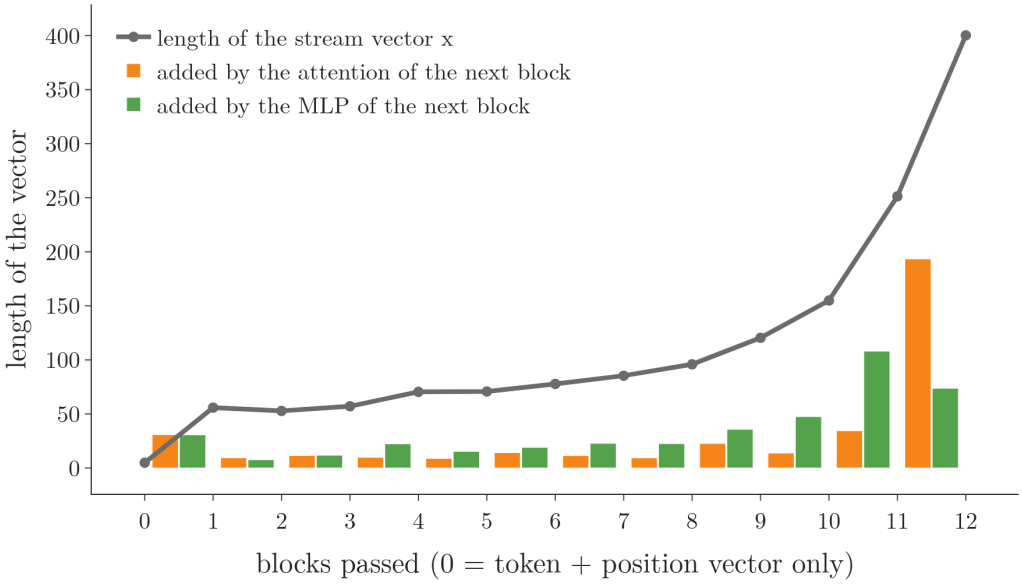


Figure 5: Length of the stream vector of " of" after each block of GPT-2 small (grey), and the lengths of the vectors added by each block's attention (orange) and MLP (green)

- The token-plus-position vector has length 4.9. Block 1 adds two vectors of length about 31 each, and the stream jumps to 56.
- From block 2 to block 10 each addition is short compared with the stream (lengths 8 to 48 against 53 to 155). The direction changes little: the cosine between the stream before and after a block (1 = same direction, 0 = at right angles) is 0.94 to 0.97.
- Blocks 11 and 12 add the largest vectors (108 from the MLP of block 11, 193 from the attention of block 12). After block 12 the length is 400.

The last LayerNorm matters. With it, the top five next tokens after "Steve Jobs was the founder of" are " Apple", " Microsoft", " the", " IBM", " Intel". Unembedding the raw stream instead (turning the vector into one score per token, section 7.1), without `ln_f`, gives " the", " a", ",", " and", ' "': the answer disappears (Notebook).

> **Extra:** Sanderson (2024, Ch 6) pictures attention as computing a change Δe that is **added** to a token's vector, nudging it towards a more specific meaning. In a prenorm model this picture is exact: the stream after a sub-layer is the stream before plus the sub-layer's output, as the formula of section 6.1 says.

6.3 Masked self-attention and the MLP

> **Key point:** The attention sub-layer is the masked multi-head attention of the earlier Notes: 12 heads of 64 in GPT-2 small. The MLP maps each **token** (G-1981)'s 768 numbers to 3,072 and back, the same for every token.

Both parts were built in earlier Notes, so here is only what GPT-2 small uses:

- **Masked multi-head self-attention:** 12 heads, each with queries, keys and values of

64 numbers. Together they make 768 numbers (see several self-attentions in parallel):

$$12 \times 64 = 768$$

The mask stops each token from reading later tokens (see masking the future). In the Notebook, every attention weight above the diagonal, a future key, is exactly 0.

- **MLP:** $768 \to 3{,}072 \to 768$, applied to each token alone (see the feed-forward network). GPT-1 already used "3072 dimensional inner states" (Radford et al. 2018, §4.1): four times the width. What the rows and columns of these two matrices do is shown in rows ask questions.

6.4 GELU

> **Key point: GPT** (G-855) replaces the ReLU of the MLP with **GELU** (G-834), a smooth curve that lets small negative values through.

Radford et al. (2018, §4.1): "For the activation function, we used the Gaussian Error Linear Unit (GELU)". Hendrycks and Gimpel (2016, §2) define it (see GELU and SiLU) as the input times the probability that a standard normal value is smaller than it. That probability is written $\Phi(x)$, the standard normal cumulative distribution function; for example $\Phi(1) = 0.841$. So:

$$\mathrm{GELU}(x) = x\,\Phi(x)$$

$$\mathrm{GELU}(1) = 1 \times 0.841 = 0.84$$

They also give a faster approximation, which is what GPT-2 uses (its configuration names it `gelu_new`):

$$u = \sqrt{2/\pi}\,(x + 0.044715x^3)$$

$$\mathrm{GELU}(x) \approx 0.5x\Big(1 + \tanh u\Big)$$

For every x from -6 to 6, the two differ by at most 0.0005 (Notebook).

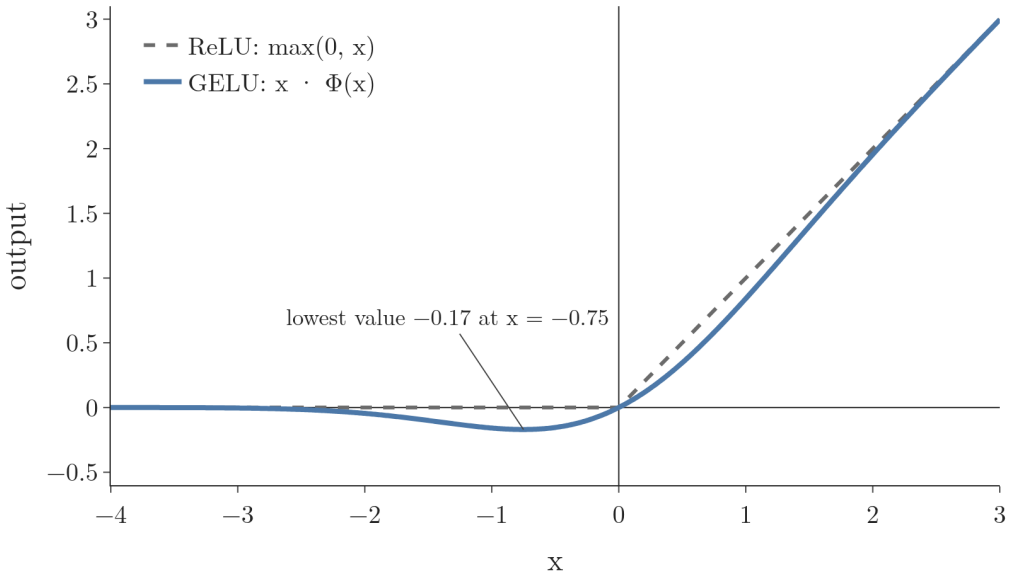


Figure 6: GELU and ReLU. For large positive x both give x; for large negative x both give about 0; near 0 GELU is a smooth curve that dips to −0.17

Figure 6 shows the difference. ReLU cuts every negative input to exactly 0 with a sharp corner. GELU passes small negative values as small negative outputs, down to −0.17 at $x = -0.75$, and its curve has no corner. In GPT-2, negative inputs are the common case: on the example prompt, 83.4 percent of the MLP's 3,072 hidden values are negative before GELU, and their outputs all lie between −0.17 and 0 (Notebook). In the words of Hendrycks and Gimpel (2016, abstract), the GELU "weights inputs by their value, rather than gates inputs by their sign as in ReLUs".

7. A guess at every position

> **Key point:** After the last block, every position's vector is turned into one score per vocabulary **token** (G-1981), so one pass gives a next-token guess at every position. Training uses all the guesses; generation uses only the last.

The vector first goes through the final LayerNorm and is then multiplied by W_E^T (W_E with rows and columns swapped), so each score is a dot product of the vector with one row of W_E.

7.1 The tied unembedding

> **Key point:** GPT-2 has no separate output matrix. The scores for the next token are the dot products of the final vector with every row of the **token embedding matrix** (G-1980) W_E.

GPT-1 wrote the output layer as follows (Radford et al. 2018, §3.1, eq. 2), with h_n the vectors after the last block, W_e the token embedding matrix and $P(u)$ the probabilities of

the next **token** (G-1981):

$$P(u) = \text{softmax}(h_n W_e^T)$$

The released GPT-2 file follows the same design: it contains `wte` and no separate output matrix (Notebook). Using one matrix for both jobs is called **tying** the weights (**tied weights**, G-1973), and the translation transformer did the same (Vaswani et al. 2017, §3.4). What these dot products mean, and how a token is chosen from them, is shown in one dot product per token.

7.2 Every position predicts

> **Key point:** On "Steve Jobs was the founder of", the same pass guesses the **token** (G-1981) after every word. After " founder" it guesses " of" (0.74, correct); after " of" it guesses " Apple" (0.69).

After	Top guess	Probability	True next token	Its probability
Steve	,	0.05	Jobs	below 0.01
Jobs	,	0.15	was	0.05
was	a	0.11	the	0.06
the	first	0.20	founder	0.01
founder	of	0.74	of	0.74
of	Apple	0.69	(not yet written)	

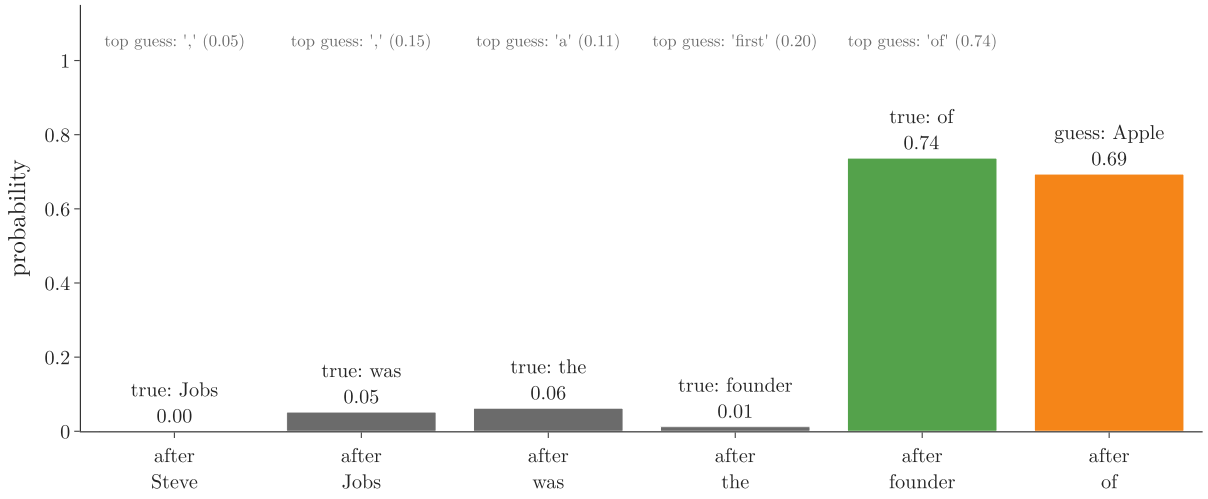


Figure 7: The table as bars: the probability the pass gave the true next token at each position (green when it was also the top guess), with the top guess above. At the last position there is no true token yet; the orange bar is the guess used for generation

Figure 7 draws the table. The early guesses are reasonable but wrong: after "Steve" alone, many continuations are possible. Each position can only see the tokens before it, which is exactly what the mask enforces. The Notebook checks it: replacing the last token " of" by " in" changes the last position's scores by up to 42, and the scores at positions 0 to 4 by exactly 0.

7.3 Training: all positions at once

> **Key point:** The training loss is the average, over all positions, of minus the log of the probability given to the true next **token** (G-1981). On a real paragraph GPT-2 small scores 3.69, against 10.83 for a uniform guess (every token equally likely).

This is the same training as in training every position in one pass: one categorical cross-entropy over the 50,257 tokens per position, averaged. The Notebook measures it on the first two sentences of the GPT-2 paper's own abstract, text published after GPT-2's training data was collected (WebText holds no links created after December 2017; Radford et al. 2019, §2.1):

Measure	Value
Tokens	61, so 60 predictions, all from one pass
Mean cross-entropy	3.69 (natural log)
Uniform guess over 50,257 tokens	$\ln 50{,}257 = 10.83$
Perplexity (G-1492), $e^{3.69}$	40.1
Top guess correct	36.7 percent

Two rows need a plain meaning:

- **Uniform guess:** a model that gives every one of the 50,257 tokens the same probability, 1/50,257. Its loss at every position is the same:

$$-\ln \frac{1}{50{,}257} = \ln 50{,}257 = 10.83$$

- **Perplexity** e^{loss}: a perplexity of 40 means the model is about as unsure as if it picked uniformly among 40 tokens. The uniform guess has perplexity 50,257.

The easiest prediction was " as" after "tasks, such" (loss 0.006, probability 0.994). The hardest were content words the context does not force, such as " begin" in "language models begin to learn" (loss 9.44).

7.4 Generation: predict, pick, append, repeat

> **Key point:** To write text, GPT keeps only the last position's guess, picks a **token** (G-1981), appends it, and runs the whole pass again on the longer text.

This is the loop of generation where the input grows by one word, without the encoder. Picking the most likely token each time (**greedy decoding**, G-870) on our prompt gives "Steve Jobs was the founder of Apple, and he . . . ": " Apple" (0.69), then "," (0.31), then " and" (0.18), then " he" (0.16).

As a check of our NumPy model, we ran greedy decoding on the prompt of the Hugging Face guide to text generation, which uses the same GPT-2 small weights (von Platen 2020). Our 30 tokens are word for word the guide's: "I enjoy walking with my cute dog, but I'm not sure if I'll ever be able to walk with my dog. I'm not sure if I'll ever be able to walk". The repetition is a known weakness of always taking the top token; **sampling** (G-1739) replaces it.

Sanderson (2024, Ch 5) frames a chatbot as this same loop run on a text that begins with a **system prompt** (G-1935), a few lines setting the scene of a user talking with a helpful assistant, followed by the user's question. The model then continues the text as the assistant would. Turning a pre-trained model into a good assistant needs further training (see from GPT-3 to ChatGPT).

8. Context size

> **Key point:** A GPT model reads at most a fixed number of **tokens** (G-1981), its **context size** (G-459): 512 for GPT-1, 1,024 for GPT-2, 2,048 for GPT-3. Anything earlier in a longer text cannot affect the prediction.

GPT-2 "increase[d] the context size from 512 to 1024 tokens" (Radford et al. 2019, §2.3); GPT-3 uses "a context window of $n_{\text{ctx}} = 2048$ tokens" (Brown et al. 2020, §2.1). In GPT-2 the limit is built into the weights: W_P has exactly 1,024 rows, so there is no position vector for a 1,025th token, and our forward pass refuses such an input (Notebook). A longer text must be cut, and the cut-off tokens cannot influence the next guess at all.

The window also sets the cost. Each attention head computes one weight for every pair of positions, n^2 of them (the cost is measured in parallel versus sequential). GPT-2 small has 12 heads in each of 12 blocks:

$$12 \times 12 = 144 \text{ heads}$$

The table multiplies the weights per head by 144:

Context n	Weights per head, n^2	All 144 heads of GPT-2 small
128	16,384	2,359,296
1,024	1,048,576	150,994,944

Context n	Weights per head, n^2	All 144 heads of GPT-2 small
2,048	4,194,304	603,979,776

Doubling the context multiplies the attention weights by 4 (Figure 8).

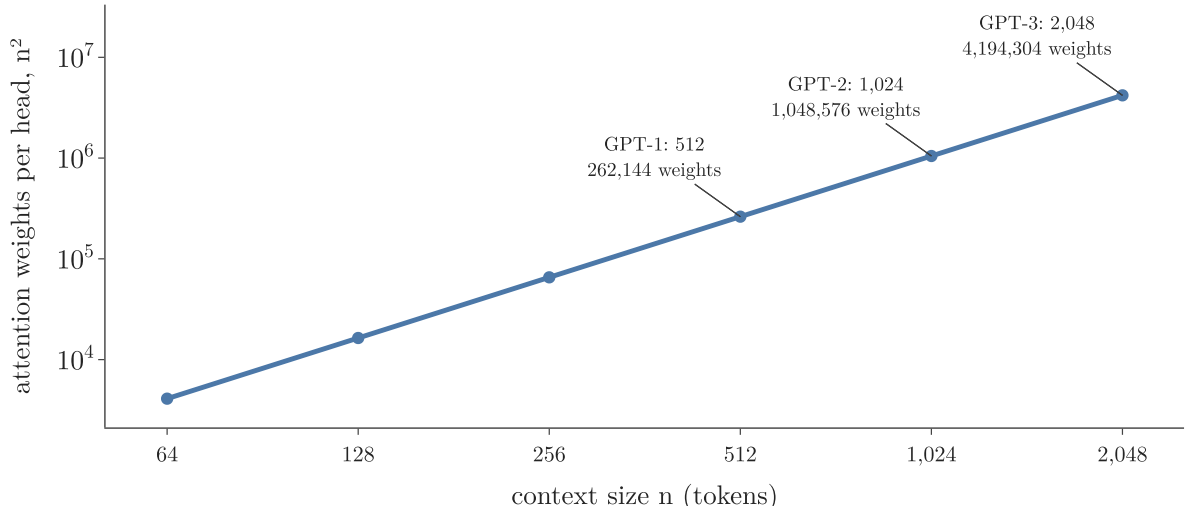


Figure 8: Attention weights computed by one head for a context of n tokens: n^2, on log scales (up the vertical axis each gridline is ten times the one below; along the horizontal axis each tick is double the one before). Each doubling of the context, as from GPT-1 to GPT-2 to GPT-3, multiplies them by 4

9. Counting the parameters

> **Key point:** GPT-2 small has 124,439,808 parameters: 31 percent in the token embedding, 23 percent in attention, 46 percent in the MLPs. In GPT-3 the embedding is a tiny share; attention holds a third and the MLPs two thirds of its 174.6 billion.

Sanderson (2024, Ch 5–7) counts GPT-3's weights one kind of matrix at a time, keeping a running tally. Figure 9 does the same for both models: GPT-2 small counted from the released file, and GPT-3 computed from the sizes in Brown et al. (2020, Table 2.1).

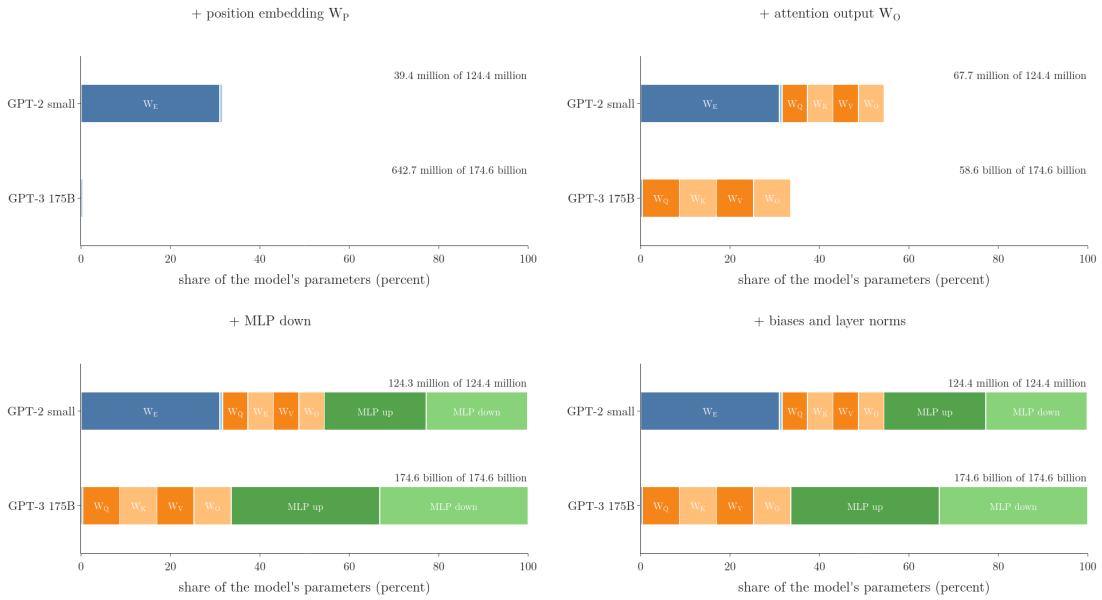


Figure 9: Running tally of parameters, one kind of matrix at a time, as a share of each model. Blue: embeddings. Orange: attention (W_Q, W_K, W_V, W_O). Green: MLP. Grey (thin): biases and LayerNorms

Watch the blue W_E: almost a third of GPT-2 small, a sliver of GPT-3. The vocabulary is the same 50,257 **tokens** (G-1981) in both, so W_E grows only with the width d, while every block grows with d^2.

9.1 GPT-2 small, matrix by matrix

> **Key point:** Each block holds $12d^2$ weights plus $13d$ biases and norm values. With $d = 768$, 12 blocks, 50,257 **tokens** (G-1981) and 1,024 positions this gives exactly the 124,439,808 of the released file.

1. **In words:** per block, attention has four $d \times d$ matrices (W_Q, W_K, W_V, W_O), and the MLP has a $d \times 4d$ matrix up and a $4d \times d$ matrix down. The small vectors of one block (biases and LayerNorm values):

Small vector	Size
biases of Q, K, V	$3d$
bias of W_O	d
bias of MLP up	$4d$
bias of MLP down	d
LayerNorm 1: gain and shift	$2d$
LayerNorm 2: gain and shift	$2d$
Total	$13d$

2. **Formula:** the embeddings, the L blocks, and the final LayerNorm ($2d$):

$$N_{\text{emb}} = Vd + n_{\text{ctx}}\, d$$

$$N_{\text{block}} = 4d^2 + 8d^2 + 13d$$

$$N = N_{\text{emb}} + L\, N_{\text{block}} + 2d$$

   In N_{block}: $4d^2$ is attention, $8d^2$ the MLP, $13d$ the small vectors.

3. **Example:** $V = 50{,}257$, $n_{\text{ctx}} = 1{,}024$, $d = 768$, $L = 12$:

Kind of matrix	Shape	Parameters	Share
Token embedding W_E (also the unembedding)	$50{,}257 \times 768$	38,597,376	31.0 percent
Position embedding W_P	$1{,}024 \times 768$	786,432	0.6 percent
W_Q, W_K, W_V, W_O (12 blocks)	768×768 each	$4 \times 7{,}077{,}888$	22.8 percent
MLP up (12 blocks)	$768 \times 3{,}072$	28,311,552	22.8 percent
MLP down (12 blocks)	$3{,}072 \times 768$	28,311,552	22.8 percent
Biases and LayerNorms		121,344	0.1 percent
Total		**124,439,808**	

The formula, term by term for $d = 768$, $L = 12$:

$$Vd = 50{,}257 \times 768 = 38{,}597{,}376$$

$$n_{\text{ctx}}\, d = 1{,}024 \times 768 = 786{,}432$$

$$L\left(12d^2 + 13d\right) = 12 \times (7{,}077{,}888 + 9{,}984)$$

$$L\left(12d^2 + 13d\right) = 85{,}054{,}464$$

$$2d = 1{,}536$$

$$38{,}597{,}376 + 786{,}432 = 39{,}383{,}808$$

$$39{,}383{,}808 + 85{,}054{,}464 = 124{,}438{,}272$$

$$124{,}438{,}272 + 1{,}536 = 124{,}439{,}808$$

The formula gives the count of the file to the last parameter (Notebook). The 12 causal-mask buffers stored in the file (`h.N.attn.bias`, $1{,}024 \times 1{,}024$ each) are not learned, so they are not counted.

9.2 GPT-3

Key point: With $d = 12{,}288$, 96 blocks and a 2,048-token context, the formula gives 174.6 billion. Attention holds 33 percent, the MLPs 66 percent, the embeddings 0.4 percent.

Brown et al. (2020, Table 2.1) give GPT-3: 96 layers, $d_{\text{model}} = 12{,}288$, 96 heads of 128, and "the feedforward layer four times the size of the bottleneck layer" (§2.1). They use "the same model and architecture as GPT-2" (§2.1), apart from "alternating dense and locally banded sparse attention patterns". The formula of section 9.1 gives the count below; it matches the paper's own count (below), so the attention patterns add no weights to the tally:

Kind	Parameters	Share
Token embedding W_E ($50{,}257 \times 12{,}288$)	617,558,016	0.35 percent
Position embedding W_P ($2{,}048 \times 12{,}288$)	25,165,824	0.01 percent
Attention, 96 blocks	57,982,058,496	33.2 percent
MLP, 96 blocks	115,964,116,992	66.4 percent
Biases and LayerNorms	15,360,000	0.01 percent
Total (tied)	**174,604,259,328**	

The count takes the output matrix as tied to W_E, as in GPT-2. The paper's compute appendix lists GPT-3 175B at 174,600 million parameters (Brown et al. 2020, Table D.1), which matches; the "175B" of the paper's title is this number rounded.

10. Summary

	Transformer decoder	GPT-2 small
Sub-layers per block	3 (masked self-attention, cross-attention, feed-forward)	2 (masked self-attention, MLP)
LayerNorm	after each addition	at each sub-layer's input, plus one after the last block
Activation	ReLU	GELU
Positions	sine–cosine	learned table, 1,024 rows
Output matrix	tied to the embeddings (Vaswani et al. 2017, §3.4)	tied: the token embedding matrix itself
Size	6 blocks, $d = 512$	12 blocks, $d = 768$, 124,439,808 parameters

- GPT is a transformer decoder without the encoder and without cross-attention, because a language model has only one input, the text so far, and predicts the next **token** (G-1981) of that same text.
- Each block adds two changes to the **residual stream** (G-1683): attention's, then the MLP's. Prenorm: LayerNorm comes before each sub-layer, which keeps the early gradients well-behaved so training needs no warm-up; the stream itself is never rescaled and grows (length 4.9 to 400); the final LayerNorm brings it back, and without it the answer " Apple" is lost.
- **GELU** (G-834) replaces ReLU, so the many negative hidden values (83.4 percent on the example) pass as small negative outputs instead of exact zeros.
- One pass gives a guess at every position, because the mask lets each position see only the tokens before it. Training averages the cross-entropy of all positions, so one text gives many training examples; generation uses the last guess, appends it and repeats.
- The **context size** (G-459) (1,024 for GPT-2) is the most text the model can read, so anything earlier cannot affect the prediction; the attention cost grows with its square, so doubling the context multiplies the attention weights by 4.
- GPT-2 small: 124,439,808 parameters, a third of them in the token embedding. GPT-3: 174.6 billion, a third in attention and two thirds in the MLPs, because the embedding grows only with the width d while every block grows with d^2.
- So **GPT** (G-855) is the decoder alone: one stack that reads a text and guesses the next token at every position in one pass.

> **Extra:** Three ideas of this Note come from Sanderson's *Transformers, the tech behind LLMs* (3Blue1Brown, 2024, Ch 5): counting the parameters matrix by matrix as a running tally, the colour rule (learned weights in blue or red, data in grey), and generation as "predict, sample, append, repeat". The figures and animations are our own design, made with the released GPT-2 small weights.

11. Sources

Built from

- Sanderson, G. (3Blue1Brown), "Transformers, the tech behind LLMs | Deep Learning Chapter 5", 2024, 3blue1brown.com/lessons/gpt, https://www.youtube.com/watch?v=wjZofJX0v4M
- Sanderson, G. (3Blue1Brown), "Attention in transformers, step-by-step | Deep Learning Chapter 6", 2024, 3blue1brown.com/lessons/attention, https://www.youtube.com/watch?v=eMlx5fFN
- Radford, A., Narasimhan, K., Salimans, T. and Sutskever, I. (2018). Improving Language Understanding by Generative Pre-Training. OpenAI. §3.1 (eq. 1, the language-model objective; eq. 2, $h_0 = UW_e + W_p$ and $P(u) = \text{softmax}(h_n W_e^T)$); §4.1 "Model specifications" (12-layer decoder-only transformer, 768 states, 12 heads, 3,072 inner states, 512-token sequences, 40,000 BPE merges, GELU, learned position embeddings).
- Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I. (2019). Language Models are Unsupervised Multitask Learners. OpenAI. §2.1 (WebText, no links after December 2017); §2.2 (byte pair encoding); §2.3 (LayerNorm moved to the input of each sub-block, extra final LayerNorm, vocabulary 50,257, context 1,024).

Other references

- Brown, T. B. et al. (2020). Language Models are Few-Shot Learners. *NeurIPS 2020*. arXiv:2005.14165 (v4). §2.1 (same architecture as GPT-2, alternating sparse attention, $d_{\text{ff}} = 4d_{\text{model}}$, $n_{\text{ctx}} = 2048$); Table 2.1 (sizes of the 8 models); Appendix D, Table D.1 (parameter counts in millions).
- Vaswani, A. et al. (2017). Attention Is All You Need. *NeurIPS 2017*. arXiv:1706.03762. §3.1 (decoder block); §3.4 (shared weights between the embeddings and the pre-softmax layer); Table 3, row (E) (learned positions nearly identical to sinusoids).
- Xiong, R. et al. (2020). On Layer Normalization in the Transformer Architecture. *ICML 2020*. arXiv:2002.04745. Abstract (Post-LN: large gradients near the output at initialisation, so warm-up is needed; Pre-LN: well-behaved gradients, trains without warm-up).
- Hendrycks, D. and Gimpel, K. (2016). Gaussian Error Linear Units (GELUs). arXiv:1606.08415. Abstract and §2 ($x\Phi(x)$, the tanh approximation).
- von Platen, P. (2020). How to generate text: using different decoding methods for language generation with Transformers. Hugging Face blog, 1 March 2020, huggingface.co/blog/how-to-generate. Greedy search section (GPT-2 output for "I enjoy walking with my cute dog").
- Weights and tokenizer: `openai-community/gpt2` (`model.safetensors`, `tokenizer.json`), huggingface.co.

12. Key terms

Terms taught in this Note come first; linked terms are recaps, taught in the Note the link opens.

Term	Meaning
GPT (G-855)	A language model that writes text by predicting the next token again and again (Generative Pre-trained Transformer); it is built from transformer decoder blocks only.
Token (G-1981)	One unit of text the model reads or writes; here, one word.
Decoder-only transformer (G-565)	A transformer with the encoder removed, so no cross-attention: a stack of blocks, each with masked self-attention and an MLP, that guesses the next token at every position. GPT is built this way.
Byte pair encoding (BPE) (G-335)	A way to build a tokenizer's vocabulary: start from single bytes and repeatedly merge the most frequent pair of symbols into a new token, so any text can be cut into tokens from a fixed vocabulary.
Token embedding matrix W_E (G-1980)	The learned table with one vector per token (50,257 vectors of 768 numbers in GPT-2); in GPT-2 also used as the output matrix.
Position embedding matrix W_P (G-1526)	A learned table with one vector per position in the text (1,024 rows in GPT-2); a token's position row is added to its token embedding, so the model knows where the token sits.
Residual stream (G-1683)	The token vector that passes from block to block, to which every sub-layer adds its output.
GELU (G-834)	A smooth activation function that multiplies each input x by $\Phi(x)$, the chance that a standard normal value is below x: $x\,\Phi(x)$ (Gaussian Error Linear Unit). GPT's MLPs use it.
Tied weights (G-1973)	Using one matrix for two jobs: the token embedding matrix turns tokens into vectors at the input, and the same matrix turns the final vector back into one score per token at the output.
System prompt (G-1935)	Text placed before the user's message that sets the scene for a chatbot.
Context size (G-459)	The largest number of tokens the model can read at once: 1,024 for GPT-2, 2,048 for GPT-3.

Term	Meaning
Masked self-attention (G-1172)	Self-attention in which each word can take information only from itself and earlier words: the later words' scores get $-\infty$ before the softmax, so their weights are 0, $\text{softmax}(QK^T/\sqrt{d_k} + M)\,V$. It lets the decoder train on whole target sentences in one pass without seeing the future.
Layer normalisation (G-1054)	Putting each observation's numbers on one scale by itself, without depending on the batch: subtract the mean and divide by the standard deviation across its own features, then scale and shift each feature by learned γ_j and β_j.
Positional encoding (G-1528)	A vector added to each word's embedding that tells the model the word's position.
Multi-head attention (G-1268)	Several self-attentions (heads) run in parallel on the same input, each with its own weights so each can capture a different kind of relation between words; their outputs are joined and mixed by one more learned matrix, W_O.
Perplexity (G-1492)	A score of how surprised a language model is by the correct tokens: the exponential of the mean cross-entropy per token; lower means it gave the correct tokens higher probability.
Greedy decoding (G-870)	Predicting by choosing the most likely token at each step and feeding it back.
Sampling (text generation) (G-1739)	Choosing the next token at random according to the softmax probabilities, instead of always taking the most probable token, so the generated text does not keep repeating itself.
Pre-LN transformer (prenorm) (G-1545)	A transformer variant with layer normalisation at the input of each sub-layer, inside the residual branch, instead of after the addition; it trains without warm-up.